

ROOM 활용

↳ ORM 라이브러리
object-relational
mapping



목차

ROOM 개요

사용 준비 및 ROOM의 주요 구성요소

Entity 의 정의

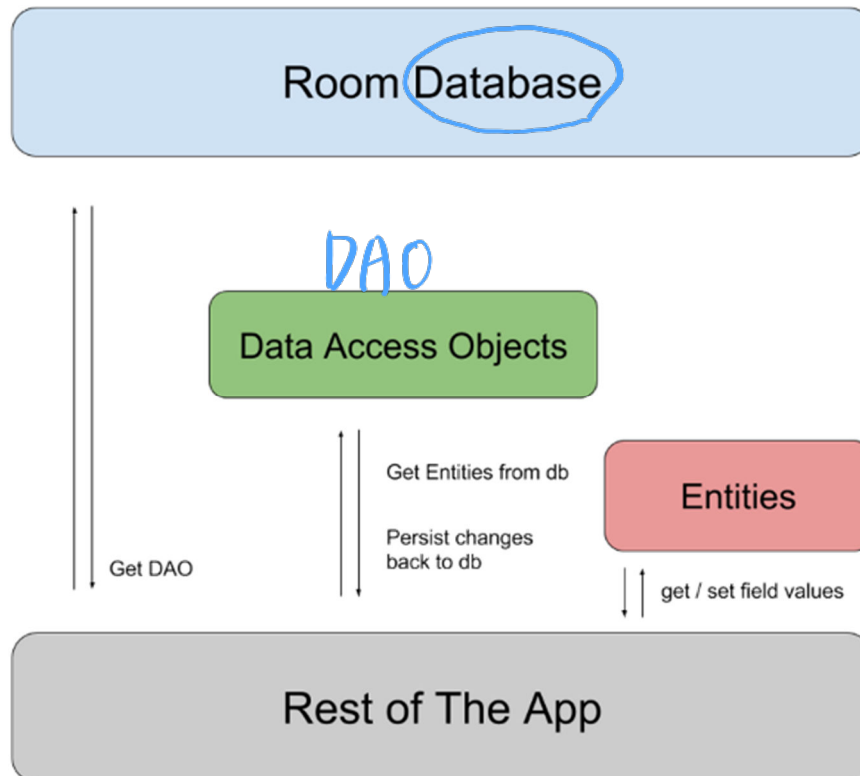
DAO(Data Access Object) 의 정의

ROOM Database의 정의

비동기 쿼리의 사용

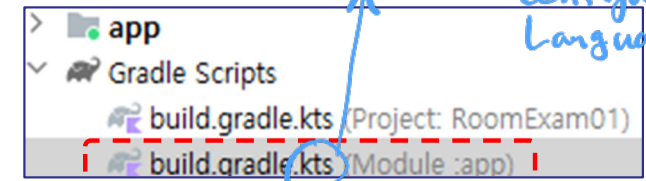
ROOM

- ◆ SQLite의 직접 사용이 아닌 추상화 계층을 적용하여 데이터베이스 활용을 지원하는 지속성 라이브러리
 - SQL 쿼리의 컴파일 시간 확인
 - Annotation (@ ...) 을 사용을 통한 상용구(Boilerplate) 코드 최소화
 - 데이터베이스 이전 간소화



• 출처: <https://developer.android.com/training/data-storage/room>

New Project 생성 시 Build Configuration Language



build.gradle 수정

- ◆ ROOM 관련 Dependency 등 추가
- ◆ 필요에 따라 비동기 처리 라이브러리 등 Dependency 추가

```
plugins {  
    ...  
    id("com.google.devtools.ksp") version "1.9.0-1.0.13" // 최신 버전을 사용  
}
```

...

```
dependencies {
```

```
    ...  
    // ROOM 관련
```

```
    val room_version = "2.6.1"  
    implementation("androidx.room:room-runtime:$room_version")  
    annotationProcessor("androidx.room:room-compiler:$room_version")
```

```
    // To use Kotlin annotation processing tool (kapt)  
    // kapt("androidx.room:room-compiler:$room_version")  
    // To use Kotlin Symbol Processing (KSP)
```

```
    ksp("androidx.room:room-compiler:$room_version")  
}
```

build.gradle(Module:...) - 프로젝트 생성 시 Kotlin DSL 선택 기준

ROOM 사용을 위해 지정

Annotation 사용을 위해 지정



주요 구성요소

ROOM Database *Open Helper와 비슷*

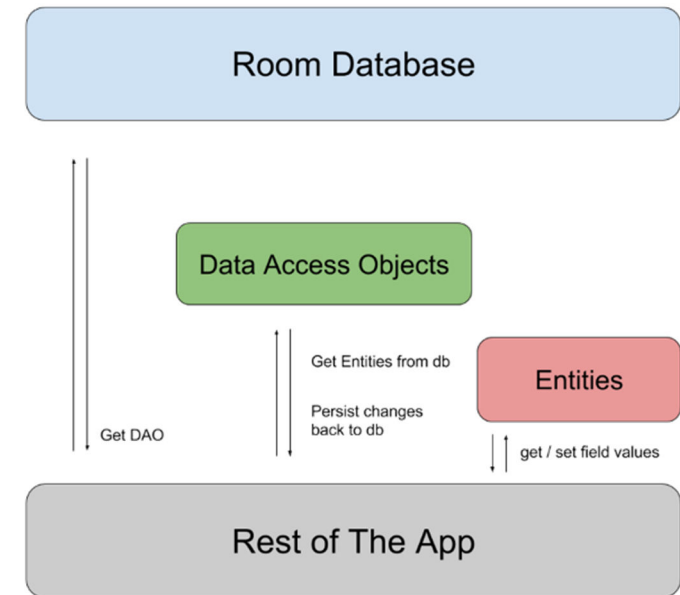
- ◆ 실제 데이터베이스를 관리
- ◆ 데이터베이스 접근점 제공
→ DB 및 DAO 생성

ROOM Entity

- ◆ 데이터베이스 상의 테이블 표현
- ◆ 결과 레코드를 저장하는 용도로도 사용
→ DTO 역할로도 사용

ROOM DAO

- ◆ Database Access Object
- ◆ 데이터베이스 테이블에 select, insert, update, 그리고 delete 를 수행하기 위한 메소드 정의 → Interface 선언만!



_id	food	country

개요

참고: <https://developer.android.com/training/data-storage/room/defining-data#kotlin>

- ◆ ROOM 지속성 라이브러리 사용 시 데이터를 표현(저장)하는 객체
- ◆ 클래스 선언과 동일한 테이블이 생성 (Annotation 사용: @...)
- ◆ 인스턴스는 생성한 테이블의 데이터 ^{레코드} 하나를 표현
- ◆ data 클래스 활용 (setter/getter, toString() 재정의 등)

•확장

테이블 명 지정

Food 테이블 생성

- ◆ Annotation 사용

•기본

```
@Entity
data class Food (
    @PrimaryKey
    val _id : Int,

    val food : String,
    val country : String
)
```

Entity 지정
테이블 명: Food

기본 키 지정

변수명으로
컬럼 생성

```
@Entity (tableName = "food_table")
data class Food (
    @PrimaryKey (autoGenerate = true)
    val _id: Int,

    @ColumnInfo(name="food")
    var food: String?,

    @ColumnInfo(name="country")
    var country: String?,

    @Ignore
    var dummy : String?
)
```

테이블 명 지정

기본 키 값
자동 생성

컬럼 명 지정

테이블 컬럼에서
제외

개요

참고: <https://developer.android.com/training/data-storage/room/accessing-data>

- ◆ ROOM 사용 시 데이터 접근 메소드를 정의하는 인터페이스
- ◆ Annotation에 따라 구현 클래스를 자동 생성
 - 편의 메소드: Query 작성 없이 삽입, 업데이트, 삭제 수행
 - 쿼리 메소드: 직접 Query 를 작성하여 테이블 접근

예

DAO 지정

편의 메소드
@Insert
@Update
@Delete

쿼리 메소드
@Query

```
@Dao
interface FoodDao {
    @Insert
    fun insertFood(vararg food: Food)

    @Update
    fun updateFood(food: Food)

    @Delete
    fun deleteFood(food: Food)

    @Query("SELECT * FROM food_table")
    fun getAllFoods(): List<Food>
}
```

@Insert

메소드에 정의한 매개변수를 테이블에 삽입

- ◆ 매개변수는 Entity 또는 Entity의 컬렉션 (예: List<Food>)
- ◆ 단일 매개변수일 경우 long 반환 (rowId)
- ◆ 컬렉션일 경우 Long[] 또는 List<Long> 반환

예

```
@Insert(onConflict = OnConflictStrategy.REPLACE)
fun insertFoods(vararg foods : Food) : List<Long>
```

테이블에 동일한 ID의 행이 있을 경우 새 항목으로 대체

가변인자: 인자
개수 유동적 지정

추가한 행의
rowId 리스트반환

```
@Insert
fun insertTwoFoods(food1: Food, food2: Food)

@Insert
fun insertFoodAndFoods(food: Food, foods: List<Food>)
```

@Update & @Delete

@Update

- ◆메소드에 정의한 매개변수 Entity 의 동일 ID 행을 찾아 수정 수행

```
@Update
fun updateFood(food: Food) : Int

@Update
fun updateFoods(vararg foods: Food)
```

수정한 행 개수
반환 가능

@Delete

- ◆메소드에 정의한 매개변수 Entity 의 동일 ID 행을 찾아 삭제 수행

```
@Delete
fun deleteFood(food: Food) : Int

@Delete
fun deleteFoods(vararg food: Food)
```

삭제한 행 개수
반환 가능

쿼리(Query) 메소드 *SELECT에만 사용(X)*

테이블의 데이터를 접근하거나 복잡한 삽입, 수정, 그리고 삭제 등을 사용할 때 사용

```
@Query("SELECT * FROM food_table")
```

```
fun getAllFoods() : List<Food>
```

Query 직접 작성

```
@Query("SELECT * FROM food_table WHERE _id = :id")
```

```
fun getFood(id: Int) : Food
```

비교

메소드의 매개변수 사용
Query 내부에 작성 시
:매개변수 형식 사용

```
@Query("SELECT * FROM food_table WHERE country IN (:country)")
```

```
fun getFoodsByCountry(country: List<String>) : List<Food>
```

컬렉션 사용 시

```
@Query("DELETE FROM food_table WHERE _id = :id")
```

```
fun deleteFoodById(id: Int)
```

Query 를 직접
작성 시 사용

ROOM Database

개요

- ◆ 데이터베이스 설정 및 데이터 접근 지점을 제공하는 추상 클래스 (RoomDatabase 상속)

- Helper 역할 수행

- ◆ DAO 생성

```
@Database (entities = [Food::class], version = 1)
abstract class FoodDatabase : RoomDatabase() {
    abstract fun foodDao() : FoodDao
}
```

생성할 테이블을 표현하는 Entity

데이터베이스 버전

테이블 접근을 위해 정의한 DAO

- ◆ 사용

```
val db : FoodDatabase = Room.databaseBuilder(
    applicationContext, FoodDatabase::class.java, "food_db"
).build()
```

Room Database 생성

생성할 DB 파일 명

```
val foodDao : FoodDao = db.foodDao()
```

DB객체에서 foodDao 생성

```
val foods = foodDao.getAllFoods()
```

DAO에 정의한 메소드 사용
insertFood(), updateFood(), deleteFood() 등

고려사항 1

Thread 의 사용

- ◆ 메인(UI) 스레드에서 ROOM DAO 메소드 사용을 제한
→ DB 접근 등 시간이 많이 걸리는 작업을 분리하도록 강제

```
fun showAllFoods() {
    val foods: List<Food> = foodDao.getAllFoods()

    for (food in foods) {
        Log.d(TAG, food.toString())
    }
}
```

실행할 DAO 메소드



Main Thread에서
실행 시 오류 발생

FATAL EXCEPTION: main
Process: ddwu.com.mobile.fooddbexam, PID: 13670
java.lang.IllegalStateException: Cannot access database on the main thread since it may potentially lock the UI for a long period of time.

- ◆ 별도의 스레드 생성 또는 비동기 쿼리 방식 사용 필요

```
binding.btnSelect.setOnClickListener{ it: View!
    Thread {
        showAllFoods()
    }.start()
}
```

Main Thread 가 아닌 새로운 Thread를
생성하여 해당 Thread에서 실행

객체를 하나만 유지하고자 할 때
적용하는 Design Pattern

RoomDatabase에 Singleton 패턴 적용

- ◆ 데이터베이스 인스턴스는 많은 자원을 소비 → 객체를 하나만 생성하여 사용할 수 있도록 구현 개선 필요

• 개선된 RoomDatabase

출처: <https://developer.android.com/codelabs/android-room-with-a-view-kotlin?hl=ko#7>

```
@Database(entities = [Food::class], version = 1)
abstract class FoodDatabase : RoomDatabase() {
    abstract fun foodDao() : FoodDao

    companion object {
        @Volatile // Main Memory 에 저장한 값 사용
        private var INSTANCE: FoodDatabase? = null

        // INSTANCE가 null 이 아니면 반환, NULL 이면 생성하여 반환
        fun getDatabase(context: Context) : FoodDatabase {
            return INSTANCE ?: synchronized(this) { // 단일 스레드만 접근
                val instance = Room.databaseBuilder(
                    context.applicationContext, FoodDatabase::class.java, "food_db"
                ).build()
                INSTANCE = instance
                instance ^synchronized // return instance
            }
        }
    }
}
```



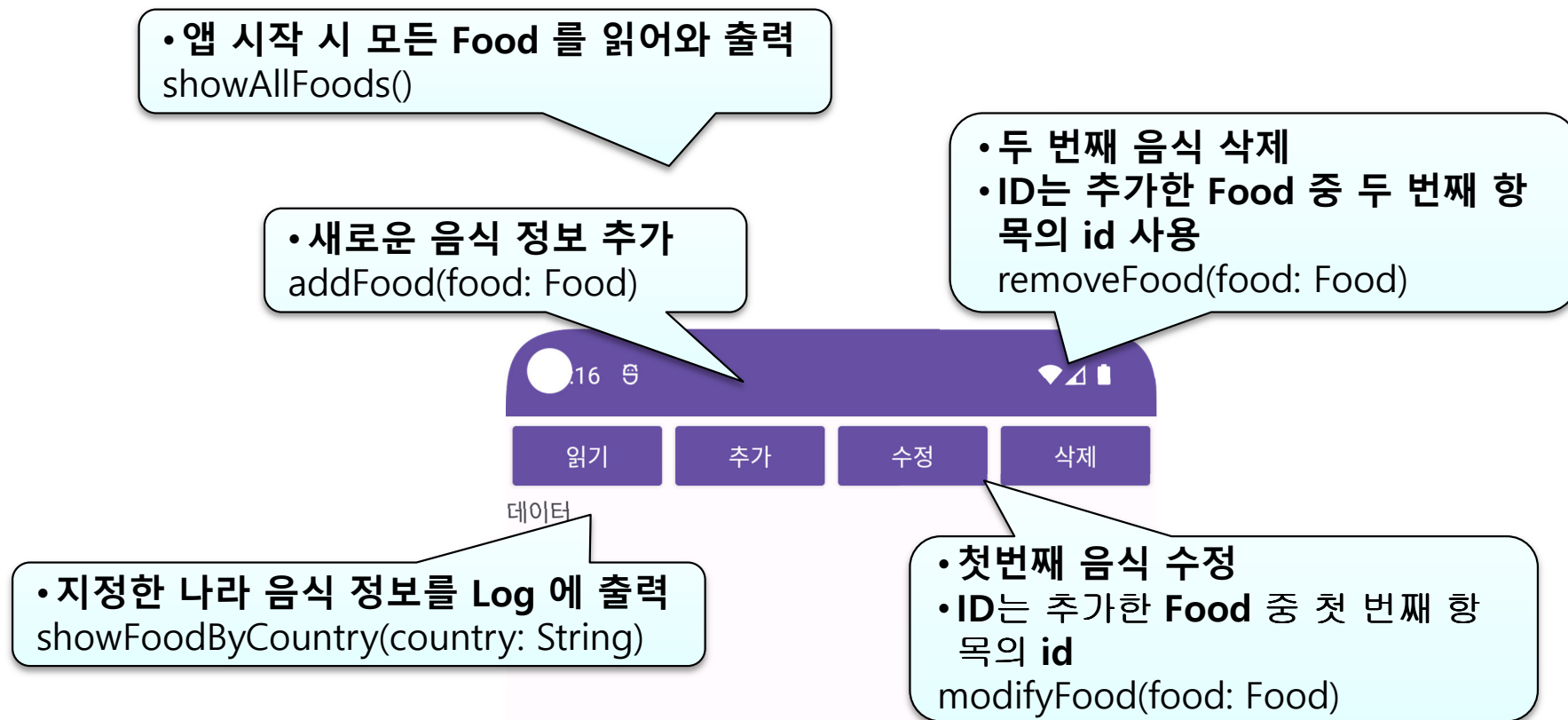
11페이지와 비교!!

• 사용 `db = FoodDatabase.getDatabase(this)`

실습 1

다음 조건에 해당하는 앱을 구현하시오.

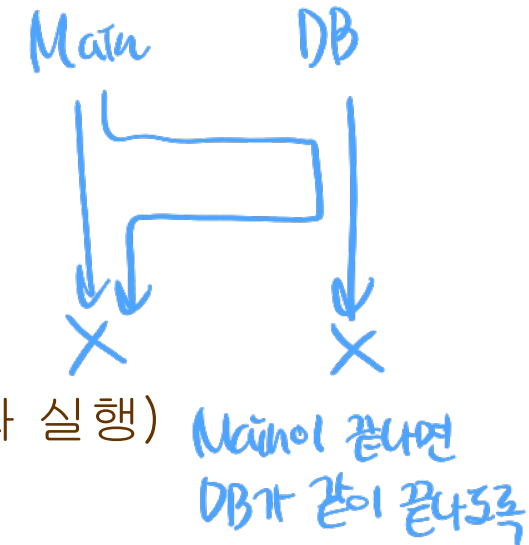
- ◆ FoodDBExam_ROOM 사용
- ◆ 실행 결과는 Log에 출력
- ◆ MainActivity 에 아래 함수에서 DAO 함수 호출



비동기(Asynchronous) 방식의 사용

❏ DAO 의 쿼리를 비동기 방식으로 구현 (고려사항 1 Thread 대체)

→ 메소드의 반환 타입을 비동기 처리 클래스 사용으로 변경



❏ 쿼리의 종류

- ◆ One-shot Write Query: insert, update, delete
- ◆ One-shot Read Query: select (한 번 실행)
- ◆ Observable Read Query: select (DB의 변경이 있을 때마다 실행)

❏ 적용 가능 언어 및 프레임워크 옵션

출처: <https://developer.android.com/training/data-storage/room/async-queries?hl=ko#options>

쿼리 유형	Kotlin 언어 기능	RxJava	Guava	Jetpack 수명 주기
원샷 쓰기	코루틴(suspend)	Single<T>, Maybe<T>, Completable	ListenableFuture<T>	해당 사항 없음
원샷 읽기	코루틴(suspend)	Single<T>, Maybe<T>	ListenableFuture<T>	해당 사항 없음
관찰 가능한 읽기	Flow<T>	Flowable<T>, Publisher<T>, Observable<T>	해당 사항 없음	LiveData<T>

↙ Thread와 동일, 더 간편하게 사용 가능!

❏ 코루틴(Coroutine) 사용 시: 4페이지 비동기 처리 dependencies 추가

비동기 원샷 쿼리의 작성

ROOM 은 Kotlin Coroutine 을 지원

- ◆ build.gradle(Module:...) dependencies 추가
implementation ("androidx.room:room-ktx:2.6.1")

한 번만 실행하는 쿼리를 통해 데이터 접근

```
@Dao
interface FoodDao {
    @Query("SELECT * FROM food_table")
    fun getAllFoods() : List<Food>

    @Query("SELECT * FROM food_table WHERE country = :country")
    suspend fun getFoodByCountry(country: String) : List<Food>

    @Insert
    suspend fun addFood(vararg food : Food)

    @Update
    suspend fun updateFood(food : Food)

    @Delete
    suspend fun removeFood(food : Food)
}
```

DAO 수정

Coroutine 사용
함수로 지정

관찰 가능한 쿼리 작성 - Flow<T>

쿼리에서 참조하는 테이블이 변경될 때마다 새로운 값을 표시 → 최신정보 유지

```
import kotlinx.coroutines.flow.Flow
```

```
@Dao
```

```
interface FoodDao {
```

```
    @Query("SELECT * FROM food_table")
```

```
    fun getAllFoods() : Flow<List<Food>>
```

- 반환타입을 Flow<T> 지정
- List<Food>의 지속적인 관찰 수행

```
    @Query("SELECT * FROM food_table WHERE country = :country")
```

```
    suspend fun getFoodByCountry(country: String) : List<Food>
```

- 일회성 쿼리 및 관찰 수행

Flow<T>의 결과 확인

- List<Food>의 변경을 지속적으로 관찰하는 Flow 지정

```
val foodFlow: Flow<List<Food>> = foodDao.getAllFoods()
```

```
foodFlow.collect { foods ->
```

```
    for (food in foods) {
```

```
        Log.d(TAG, food.toString())
```

```
    }
```

```
}
```

- collect ()를 사용하여 Flow의 변경을 수집하여 확인

collect는
suspend 함수로
지정되어 있다.



Coroutine 함수의 호출

Scope 지정

출처: <https://developer.android.com/kotlin/coroutines/coroutines-adv?hl=ko#coroutinescope>

- ◆ Coroutine 함수(suspend 함수, Flow 사용 등)들은 코루틴 스코프 내에서 실행
- ◆ Coroutine 의 실행 범위와 수명주기 관리
- ◆ CoroutineScope(Dispatchers.IO).launch {...} 사용
 - 특정 작업의 수명주기에 따라 Coroutine 수행
 - Dispatchers.IO: 네트워크, DB접근, 또는 파일 입출력 작업에 사용
→ 입출력에 최적화 된 스레드 풀 사용

예

```
fun addFood() {
    val food = Food(0, "된장찌개", "한국")
    CoroutineScope(Dispatchers.IO).launch { this: CoroutineScope
        foodDao.insertFood(food)
    }
}
```

• suspend 함수 호출
• Flow<T> 반환 함수 호출
등에도 적용

Main, default
(화면 갱신)
(CPU 파워가 필요할 때)

return 값이 없을 때

고려사항

Flow<T> 사용 시의 고려사항

- ◆ Flow<T> 는 지속적으로 대상을 관찰하므로 변경이 있을 때 마다 쿼리가 다시 실행됨
- ◆ 관심 결과와 관계 없이 테이블의 변경이 있을 때마다 실행되므로 불필요한 실행 발생
→ Flow<T>.distinctUntilChanged() 사용 : 변경이 있을 때만 쿼리를 실행하도록 변경한 Flow<T> 반환

기존 Flow<T> 사용의 개선

- 데이터
- 변경이 있을 때만 쿼리를 수행하는 Flow 로 변경

```
val foodFlow: Flow<List<Food>> = foodDao.getAllFoods()
foodFlow.distinctUntilChanged().collect{ foods ->
    for (food in foods) {
        Log.d(TAG, food.toString())
    }
}
```

실습 2

 다음 조건을 충족하는 앱을 구현 하시오.

- ◆ RoomExam01 사용
- ◆ 비동기 쿼리 방식으로 변경
- ◆ 전체 음식 정보는 RecyclerView에 표시 → 앱 실행 시 표시
- ◆ 변경(추가/수정/삭제) 시 RecyclerView 에 자동으로 변경사항 반영 → Flow<T> 사용
 - Hint: collect 수행 시 마다 새로운 list 를 adapter에 설정 후 notify
- ◆ 수정/삭제 수행 시 음식 이름을 기준으로 수행

